

TEORIA
PSP



MIKEL CLAVERO

2025-26

AURKIBIDEA

1 TCP/IP PROTOKOLO-PILA.....	3
1.1 TCP/IP GERUZAK.....	3
1.2 SARBIDE FISIKO GERUZA.....	4
1.3 INTERNET GERUZA.....	4
1.4 GARRAIO GERUZA.....	4
1.5 APLIKAZIO GERUZA.....	5
2 SOCKET KONTZEPTUA.....	5
2.1 HOSTEN IDENTIFIKAZIOA.....	6
2.2 TCP KONEXIOA.....	6
2.3 UDP KONEXIOA.....	7
3 C# ETA SOCKET-AK.....	7
4 ASINKRONIA.....	8
4.1 async/await.....	8
5 SARE ZERBITZUAK.....	9
6 HYPERTEXT TRANSFER PROTOCOL (HTTP).....	10
6.1 C# KLASEAK.....	10
6.2 HTTP API REST.....	10
6.3 REST APLIKAZIOAK.....	11
6.4 JSON.....	11
7 SEGURTASUNA.....	12
7.1 KRIPTOGRAFIA.....	12
7.1.1 ZIFRATZE SIMETRIKOA.....	12
7.1.2 ZIFRATZE ASIMETRIKOA.....	12
7.2 ZIURTAGIRIAK.....	12
7.3 SSL/TLS.....	13
7.4 HASH FUNTZIOAK.....	13

1 TCP/IP PROTOKOLO-PILA

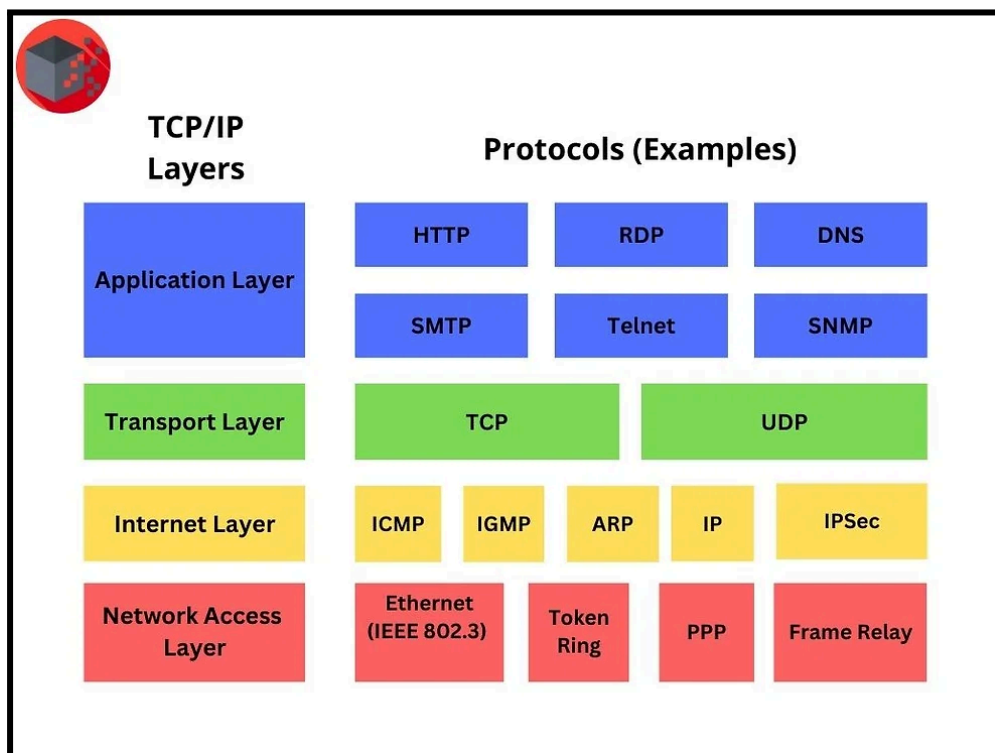
Interneten, TCP/IP protokolo-pila erabiltzen da konexioak ezarri eta egiteko, eta hainbat maila edo geruzatan antolatutako protokolo-multzo batean oinarritzen da. Interneten oinarria da eta bi osagai nagusi ditu (baina ez bakarrak): TCP (Transmission Control Protocol), eta IP (Internet Protocol).

1.1 TCP/IP GERUZAK

Esan dugunez, TCP/IP protokolo-pila 4 geruzatan sailkatzen ditu protokoloak:

1. Sarbide fisikoa: Sareko hardwarearekin komunikatzen da.
2. Internet: Datuak nodoen artean bideratzen ditu.
3. Garraioa: Datuen fidagarritasuna eta segmentazioa kudeatzen du.
4. Aplikazioa: Erabiltzailearen eta zerbitzuen arteko elkarrekintza.

Geruza bakoitza bere azpian edo gainean daukan geruzekin komunikatzen da.



1.2 SARBIDE FISIKO GERUZA

Datuen transmisioa hardware mailan kudeatzen duen geruza da. Bere funtzio nagusia bitak medio fisikoen bidez nola bidaltzea da, hala nola kableak, zuntz optikoak edo haririk gabeko medioen biartez. Modulazioa, seinalizazioa eta transmisio abiadurak definitzen ditu, sarean gailuek fisikoki komunikatu ahal izateko. Gure ordenagailuan, sare-txartela (Ethernet, WiFi...) betetezen oi du eginkisun hau.

1.3 INTERNET GERUZA

Datuen bideratzea kudeatzen duen geruza da. Bere funtzio nagusia datuak nodoen artean helaraztea da, IP (Internet Protokoloa) erabiliz. Bideratze-protokoloek (adibidez, IPv4 eta IPv6) datuak helmugara iristeko bidea zehazten dute, sare ezberdinen artean komunikazioa ahalbidetuz. Geruza honetan gehien bat Routerrek egiten dute lan, ordenagailuak helmudak direlarik.

IP helbidea (Internet Protokoloa) ordenagailu, gailu edo nodo baten identifikazioa da sare batean. Mezuak nori bidaltzen diren jakiteko ezinbestekoa da. IP helbideak bi motatakoak izan daitezke:

- IPv4: 32 bitko helbideak (adibidez, 192.168.1.1)
- IPv6: 128 bitko helbideak (adibidez, 2001:0db8:85a3:0000:0000:8a2e:0370:7334)

1.4 GARRAIO GERUZA

Garraio geruza TCP/IP protokolo-pilan datuen transferentzia kudeatzen duen geruza da. Datuak segmentuetan antolatzen ditu (zatiak), eta bere funtzio nagusia datuen fidagarritasuna eta segurtasuna bermatzea da. Hau da:

1. Bidaltzerakoan datuak zati txikiagotan zatitzen dira eta saretik bidaiatzen dute.
2. Heltzerakoan garraio geruzak zati horiek berriro elkartzen ditu.

Garrario geruzan bi protokolo nagusi aurkitzen ditugu:

- TCP (Transferentzia Kontrol Protokoloa): Datuen transferentzia fidagarria eta ordenatua ziurtatzen du, berrespenak eta errore zuzenketak erabiliz. Adibidez: HTTP, FTP, SSH...
- UDP (User Datagram Protocol): Datuen transferentzia azkarragoa eskaintzen du, baina fidagarritasuna eta ordena bermatu gabe, adibidez, audio eta bideo transmititzailerako. Adibidez: VoIP, DNS...

Garrario geruzaren beste funtzio garrantzitsu bat informazioa aplikazio egokira bideratzea da. Hori portu-zenbakien bidez egiten da. Adibidez, HTTP zerbitzarira badoa mezua, 80 atakara bidaliko du. Hau da, portu-zenbaki baten atzean beti egongo da aplikazio bat.

Atakak 16 bit dituen zenbaki baten bidez identifikatzen dira, 0 - 65535 bitartekoa, non lehenengo 1023ak erreserbatuta dauden. Erreserbatutako portu zenbakiak kontsultatzeke:

- Linux: /etc/services
- Windows: C:\Windows\System32\drivers\etc\services.

1.5 APLIKAZIO GERUZA

Erabiltzaileari zuzenean zerbitzuak eskaintzen dizkion geruza da. Bere funtzio nagusia aplikazioek sarean elkarri datuak bidali eta jasotzeko aukera ematea da. Geruza honetan daude sareko komunikazio-protokolo espezifikoak, adibidez:

- HTTP/HTTPS: Web orrialdeak arakatzeko.
- FTP: Fitxategiak transferitzeko.
- SMTP: Mezu elektronikoak bidaltzeko.
- DNS: Domeinu-izenak IP helbideetara itzultzeko.

Bezero-zerbitzari eredua sareko arkitektura tipiko bat da, non:

- Bezeroa: Zerbitzuak edo baliabideak eskatzen ditu. Adibidez, nabigatzaile bat irekitzen duzunean eta web orri bat bisitatzen duzunean, nabigatzailea bezero bezala jotzen du.
- Zerbitzaria: Bezeroek eskatzen dituzten zerbitzuak edo baliabideak eskaintzen dituena da. Adibidez, web zerbitzari batek web orriak gordetzen eta entregatzen ditu bezero batek eskatzen dituenean.

2 SOCKET KONTZEPTUA

Socket bat banatuta dauden bi prozesuren arteko komunikazio bidirekzionala ahalbidetzen duen konektorea da. Beraz, socket-ek sare baten bidez komunikatzen diren bi programen arteko komunikazio bidirekzionala ahalbidetzen dute.

Socket batek zehaztuta eduki behar ditu:

- IP Helbidea: sarean identifikatzeko.
- Portu-zenbakia: programa identifikatzeko.

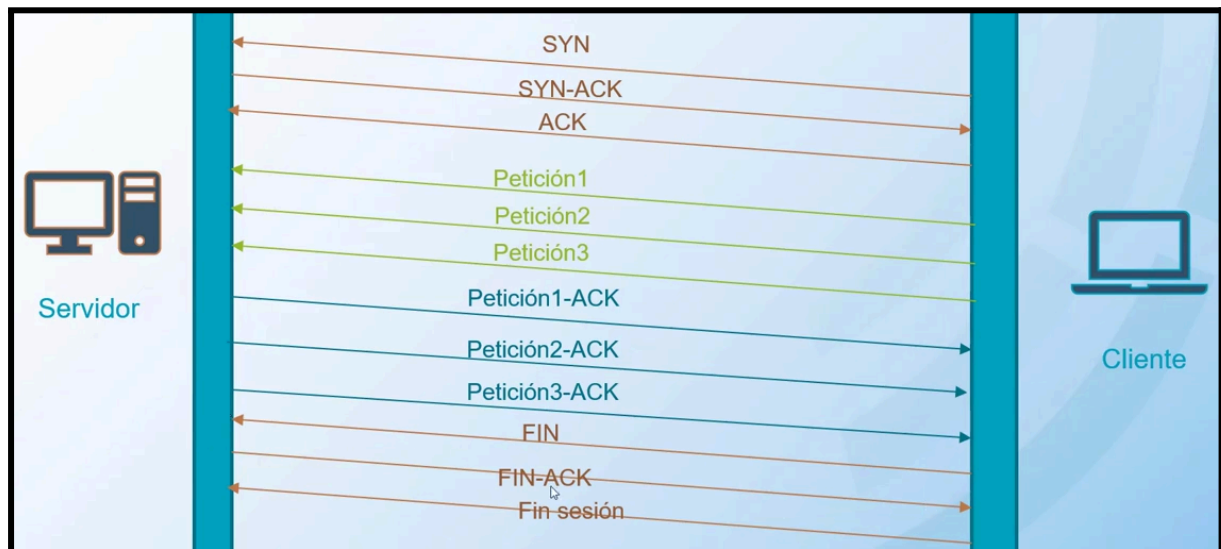
2.1 HOSTEN IDENTIFIKAZIOA

ZERBITZARIA	BEZEROA
Ezaguna den IP helbidea	IP helbidea Sistema Eragileak esleitzen dio
Saretik komunikatu nahi diren programak	Portu-zenbakia SEak esleitzen dio
	Komunikazioa hasten du eskaera eginez

2.2 TCP KONEXIOA

TCP protokoloa erabilena da:

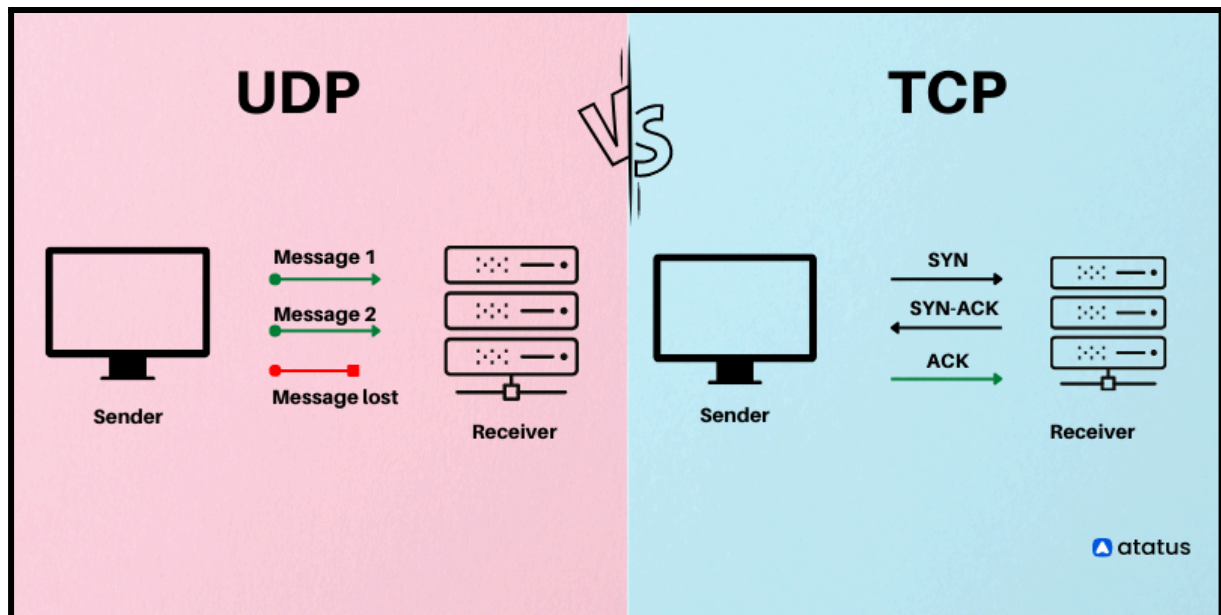
- Paketeen entregatzea bermatzen du.
- Paketeen ordena errespetatzen du.
- Konexio fiable bat erabiltzen du.
- TCP erabiltzen duten protokolo adibidea: HTTP, HTTPS, FTP, SMTP...
- Konexioa hasteko 3 way handshake deitzen den 3 segmentuen bidalketa egiten da.



2.3 UDP KONEXIOA

UDP fidagarritasuna behar ez denean erabiltzen da:

- Ez du konexio bat ezartzen komunikatzeko.
- Ez du paketeen entregatzea bermatzen.
- Ez du paketeen ordena errespetatzen.
- Fidagarritasuna txikiagoa baina abiadura handiagoa.
- Erabilera adibideak: streaming, VoIP, bideojokoak...



3 C# ETA SOCKET-AK

Socket-ak kudeatzeko ondorengo klaseak erabiliko ditugu:

- TcpListener: TCP konexioak entzuteko socketak.
- TcpClient: bezero konexioak kudeatzeko socketak.
- NetworkStream: socketak erabilia datuak bidali eta jasotzeko.

Datuak era eroso baten bidali eta irakurtzeko (kontsolatik bezala):

- StreamReader
- StreamWriter

4 ASINKRONIA

Konkurrentzia lortzeko gaur egun asko erabiltzen den kontzeptua: asinkronia.

Erabil dezakezu (eta erabili behar duzu) zure kodetik kanpoko baliabide bat kontsumitzen baduzu: fitxategi bat, datu-base bat, online zerbitzu bat...

Task edo Thread klaseekin lortu daiteke, baina asko errazten du implementazioa.

4.1 async/await

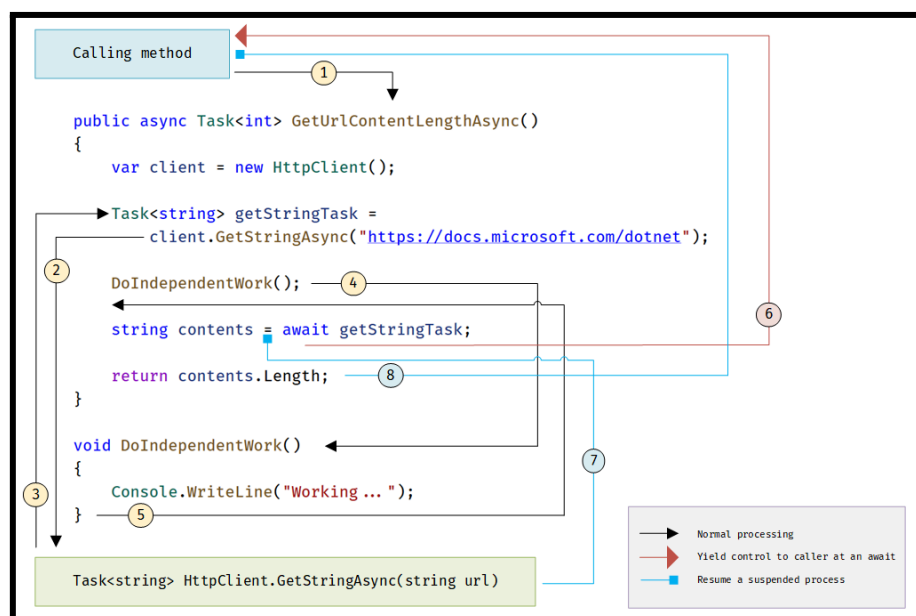
C# lengoian async eta await hitzak erabiliko ditugu kodigo asinkronoa idazteko:

- **async**: metodo batek operazio asinkronoak eduki ditzakeela adierazteko.
- **await**: async deklaratu den metodo baten barruan deitu daiteke, ataza bat (Task) bete arte itxaroten du, baina programa blokeatu gabe, exekuzio kontrola async metodoa deitu duenari itzultzen dio.

ADI: Metodo bat async bezala deklaratzek ez du asinkronia lortzen, haren implementazioan await bat egon behar du, bestela metodo sinkrono normal bat bezala exekutatu da.

Metodo asinkrono batek ondorengoetarikoa itzulera balioa bat eduki beharko du:

- **Task**: ez duenean itzulera balio bat edukiko.
- **Task<T>**: itzulera balio bat edukitzen duenean (return hitzaren bitartez).
- **void**: geratera kontrolatzaileerako (kasu berezia).



- Errekurtsuen erabilpen eraginkorra: async/await-ek aukera ematen du haria aske uzteko E/S eragiketa bat itxaroten den bitartean (adibidez, AcceptTcpClientAsync), haria blokeatu gabe. Honek baliabide gutxiago erabiltzen ditu, hariak ez direlako geldirik geratzen eta beste zeregin batzuk artatzeko erabil daitezkeelako.
- Eskalagarritasuna: Zerbitzari asinkronoak konexio kopuru handia kudeatu dezake bezero bakoitzarentzat hari berri bat sortu beharrik gabe.
- Irakurgarritasuna eta mantentzea: async/await-en erabilerak kode argiagoa eta errazago mantentzekoa ahalbidetzen du.

5 SARE ZERBITZUAK

Sare baten bidez funtzionalitate espezifikoak ematen dituzten aplikazioak edo prozesuak. TCP/IP protokolo-pilaren goiko geruzan kokatzen dira. Haien ezaugarri nagusiak:

- Zerbitzari-bezero komunikazio modelo jarraitzen dute gehienetan.
- Sare protokolo espezifikoak erabiltzen dute: FTP, SMTP, HTTP, DNS...
- Interoperabilitatea: sistema ezberdinak komunikatu ahal dituzte (Windows, Unix...).
- Asinkronia: hainbat eskaera aldi berean kudeatu ahal dituzte.
- Segurtasuna: sareen arriskuak saiesteko, segurtasun mekanismoak erabiltzen dituzte.
- Jada programaturik dauden aplikazio asko existitzen dira.



6 HYPERTEXT TRANSFER PROTOCOL (HTTP)

- Bi ordenagailuren artean “hipertestu”aren transferentzia aurrera eramateko.
- Web orrialde baten edukiak HTML lengoaia erabilita kodifikaturik daude.
 - Beste eduki batzuk: irudiak, css eta javascript.
- 80 portu-zenbakian entzuten du.
- Datuak zifratuta bidaltzeko HTTPS protokoloa: 443 portu-zenbakia.
- Zerbitzari aplikazio adibideak: Apache2, Nginx, IIS...
- Bezero aplikazio adibideak: Chrome, Firefox, Edge...

6.1 C# KLASEAK

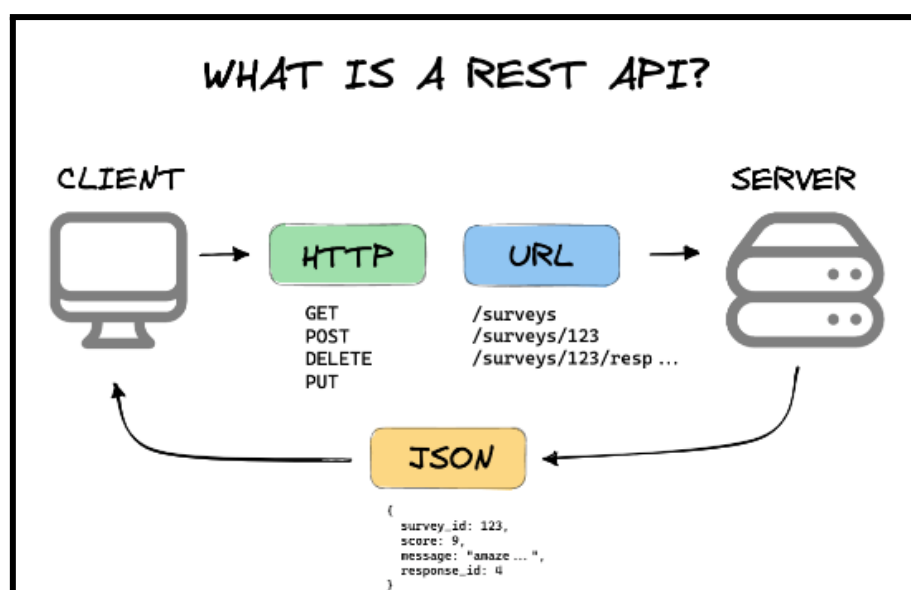
HttpClient klasea erabiliko dugu. HTTP eskaera bate egiteko:

- HttpClient bezeroa = new HttpClient();
- HttpResponseMessage erantzuna = bezeroa.GetAsync(“URL”)

Erantzuna HttpResponseMessage objektu bat izango da, non:

- // HTTP erantzun kodea itzuliko du (200, 404...).
- response.StatusCode
- // HTTP erantzunaren edukiak String moduan itzuliko ditu.
- string edukiakString = response.Content.ReadAsStringAsync()

6.2 HTTP API REST



6.3 REST APLIKAZIOAK

REST (Representational State Transfer) aplikazio web batek, normalean, CRUD (Create, Read, Update and Delete) operazioei erantzuten die, ondorengo HTTP metodoekin: GET, POST, PUT eta DELETE.

Metodo hauek ondo erabiltzen baditugu (hau da, estandarra jarraitzen badugu) aplikazio koherenteagoak, seguruagoak eta ulergarriagoak sortuko ditugu.

- 200 OK: Eskaera arrakastatsua izan da eta zerbitzariak datuak bueltatzen ditu.
- 201 Created: Eskaera ondo burutu da eta baliabide berri bat sortu da.
- 204 No Content: Eskaera ondo gauzatu da, baina ez dago bueltatzeko edukirik.
- 400 Bad Request: Bezzeroaren eskaeran akats bat dago, datuak okerrak dira edo faltan daude.
- 401 Unauthorized: Bezzeroa ez dago autentifikatuta edo saioa hasi gabe dago.
- 403 Forbidden: Bezzeroa autentifikatuta dago, baina ez dauka baliabidera sartzeko baimenik.
- 404 Not Found: Eskatutako baliabidea ez da aurkitu zerbitzarian.
- 405 Method Not Allowed: Erabilitako HTTP metodoa (GET, POST, etab.) ez dago baimenduta.
- 500 Internal Server Error: Zerbitzariak errore bat izan du eskaera prozesatzean.
- 503 Service Unavailable: Zerbitzaria une honetan ez dago eskuragarri.

6.4 JSON

Gaur egun, normalean, API REST erantzunak JSON formatua erabiltzen dute. JSON JavaScript Object Notation esan nahi du, eta datuak gordetzeko eta trukatzeko sintaxi bat da.

JSON formatua testuan oinarritutako formatua denez, zerbitzari batera eta zerbitzari batetik erraz bidal daiteke, edozein programazio-lengoaiek datu-formatu gisa erabil dezakelarik.

JSON sintaxi-arauak ondorengoak dira:

- Datuak izen/balio bikoteetan daude.
- Datuak komaz bereizten dira.
- Giltzek objektuak gordetzen dituzte.
- Kortxeteek array-ak gordetzen dituzte

7 SEGURTASUNA

7.1 KRIPTOGRAFIA

Informazio irakurgarria informazio irakurtezin bihurtzeko zientzia da. Motak:

- Simetrikoa
- Asimetrikoa

7.1.1 ZIFRATZE SIMETRIKOA

Zifratzeko eta deszifratzeko gako berbera erabiltzen den kriptografia da. Gakoa igorleak zein hartzaileak aldezturik ezagutu behar dute.

- Adibide klasikoa: Enigma makina (2. mundu gerra).
- Arazorik handiena: hirugarrenak gako bakarra atzeman dezakete.
- Algoritmoak: DES, IDEA, RC2, Blowfish, 3DES.

7.1.2 ZIFRATZE ASIMETRIKOA

Gakoen erabilera: publikoa zifratzeko eta pribatua deszifratzeko.

Gako pribatua bakarrik ezagutzen du gako horren jabeak.

Gako publikoa edozeinek jakin dezake.

- Algoritmoak: DSA, RSA.
- Protokolo tipikoa: PGP.

7.2 ZIURTAGIRIAK

Ziurtagiri digitala entitate baten identifikazio elektroniko gisa funtzionatzen duen fitxategi bat da (adibidez, webgune, pertsona edo enpresa bat).

Ziurtagiri honek gako publikoak ditu eta Ziurtagiri Agintaritza (CA) konfiagarri batek sinatzen du. Bere helburu nagusia segurtasuna eta fidagarritasuna bermatzea da komunikazio elektronikoetan.

Ohiko formatuak:

- X.509: Ziurtagiri digitalen estandarra. SSL/TLS protokoloetan erabiltzen da.
- PKCS#12 (PFX): Formatoa, ziurtagiria eta gako pribatua batzen dituena.
- PEM (Privacy Enhanced Mail): ASCII formatuan, ziurtagiriak esportatzeko ohikoa.

7.3 SSL/TLS

SSL/TLS gaur egun informazioa era seguruan partekatzeko protokolo nagusia da. Adibidez, HTTPS protokoloak erabiltzen du. Funtzionamendua:

1. Zerbitzariak ziurtagiria bidaltzen dio bezeroari.
2. Bezeroak ziurtagiria zuzena dela egiaztatzen du.
3. Ziurtagirian dagoen gako publikoaren bitartez (zifratze asimetrikoa), bezeroak klabe bat adosten du zerbitzariarekin era seguru baten.
4. Informazio bidalketa: hemendik aurrera, adostutako klabea erabiliz informazioa zifratuko dute era simetriko baten (azkarragoa delako).

Hau da, hasierako prozesuan (HandShake) zifratze asimetrikoa erabiltzen da klabe bat adosteko era dinamiko baten. Informazioa bidalketan, berriz, zifratze simetrikoa erabiltzen da. Konexio bakoitzean klabe berri bat adostuko da.

7.4 HASH FUNTZIOAK

Hash funtzio batek datu-sarrera bat emanda balio bakar eta finko bat sortzen du. Sortzen duen balio edo gako honek ondorengo ezaugarriak dauzak:

- Luzera finkoko irteera sortzen du.
- Ezinezkoa da hash kode batetik dagokion hash mezua lortzea.
- Mezu bat kontutan hartuta, ezin da hash kode berekorik aurkitu.